# Image Classification on Mini-ImageNet: A CNN Design Study and Occlusion-Sensitivity Analysis

***Deep Learning – Module 4 Assignment***

***Gitarani Mohanty***

## 1. Introduction

This report studies how the main design choices of a convolutional neural network (CNN) affect image-classification accuracy (Part A) and then probes the best model with an occlusion-sensitivity experiment to find out whether it recognises the object itself or relies on surrounding context (Part B, after Zeiler & Fergus, 2014). Part A varies the number of convolutional layers, the number of fully connected (FC) layers, the number of filters, max-pooling, stride and training time, and then tries to maximise accuracy. It ends with an analysis of mis-classified images.

## 2. Dataset and experimental setup

**Data.** The run reported here used a synthetic stand-in dataset: 20 classes × 60 images, 84 × 84 RGB. Each class is a combination of a shape (circle, square, triangle, stripes) and a colour (five hues), drawn at random size and position on a noisy random-colour background. A stratified split gives 959 training, 121 validation and 120 test images. Real Mini-ImageNet has 100 classes × 600 images at the same resolution. Pixels are normalised with the per-channel mean and standard deviation of the training split. No data augmentation is used in the ablation study, so differences come only from the studied factor.

**Network family.** Each convolutional stage is Conv(3×3, padding 1) → BatchNorm → ReLU, optionally followed by 2×2 max-pooling. The feature map is reduced to 4 × 4 by adaptive average pooling (so the FC input size is the same for every configuration), followed by FC layers with ReLU and dropout 0.3. The **baseline** has 3 conv layers (32-64-128 filters, stride 1) with max-pooling and one hidden FC layer of 256 units (0.62 M parameters).

**Training protocol.** AdamW (learning rate 1e-3, weight decay 5e-4), batch size 128, cosine learning-rate decay, mixed precision, 15 epochs, random seed 42, on a Tesla T4 GPU. One factor is changed at a time while everything else stays at the baseline value. The checkpoint with the best **validation** accuracy is kept, and the **test** set is used only for reporting (top-1 and top-5 accuracy).

**How much to trust the numbers.** The test set has only 120 images, so one image is 0.83 percentage points and the binomial standard error at about 60 % accuracy is roughly 4.5 points. Each configuration was trained once with a single seed. Differences smaller than about 5–9 points between two rows should therefore not be over-interpreted; only large effects are reliable.

# 3. Part A – Effect of the network parameters

## 3.1 Results table

| Factor | Configuration | Params (M) | Val acc (%) | Test top-1 (%) | Test top-5 (%) |
|---|---|---|---|---|---|
| **Baseline** | 3 conv (32-64-128), 2 FC, pool, stride 1, 15 ep | 0.62 | 64.46 | 61.67 | 95.83 |
| Conv layers | 1 conv (32) | 0.14 | 33.06 | 31.67 | 92.50 |
| | 2 conv (32-64) | 0.29 | 47.93 | 51.67 | 92.50 |
| | 4 conv (32-64-128-256) | 1.44 | 67.77 | 67.50 | 99.17 |
| | 5 conv (32-64-128-256-256) | 2.03 | 70.25 | 69.17 | 99.17 |
| FC layers | 1 FC (output only) | 0.13 | 64.46 | 62.50 | 96.67 |
| | 2 FC, wide (1024) | 2.21 | 64.46 | 61.67 | 96.67 |
| | 3 FC (512-256) | 1.28 | 61.16 | 53.33 | 97.50 |
| | 4 FC (512-256-128) | 1.31 | 54.55 | 55.00 | 97.50 |
| Filters | 16-32-64 | 0.29 | 58.68 | 60.83 | 97.50 |
| | 64-128-256 | 1.43 | 65.29 | 56.67 | 97.50 |
| | 128-256-512 | 3.58 | 58.68 | 53.33 | 96.67 |
| | 64-64-64 (constant) | 0.34 | 64.46 | 60.83 | 96.67 |
| | 128-64-32 (decreasing) | 0.23 | 61.16 | 64.17 | 99.17 |
| Max-pooling | no max-pooling | 0.62 | 60.33 | 52.50 | 94.17 |
| Stride | 2 (first conv) | 0.62 | 66.12 | 62.50 | 98.33 |
| | 3 (first conv) | 0.62 | 65.29 | 58.33 | 98.33 |
| | 2 (all convs), no pooling | 0.62 | 58.68 | 57.50 | 95.83 |
| Epochs (one 30-epoch run, constant LR) | after 5 epochs | 0.62 | 54.55 | 51.67 | 92.50 |
| | after 10 epochs | 0.62 | 60.33 | 59.17 | 95.83 |
| | after 15 epochs | 0.62 | 60.33 | 65.00 | 97.50 |
| | after 20 epochs | 0.62 | 69.42 | 74.17 | 96.67 |

| Factor | Configuration | Params (M) | Val acc (%) | Test top-1 (%) | Test top-5 (%) |
|---|---|---|---|---|---|
| | after 25 epochs | 0.62 | 71.07 | 79.17 | 97.50 |
| | after 30 epochs | 0.62 | 78.51 | 84.17 | 99.17 |
| Improved models (40 ep, augmentation, label smoothing, dropout 0.5) | A: 4 conv (64-128-256-512) | 5.76 | 66.12 | 59.17 | 100.00 |
| | B: 6 conv, VGG-style (64-128-256, 2 convs per stage) | 3.25 | 68.60 | 73.33 | 100.00 |
| | C: 8 conv, VGG-style (64-128-256-512, 2 per stage) | 8.89 | 71.07 | 70.83 | 100.00 |

The "epochs" rows read the accuracy of one run after n epochs. The checkpoint chosen on validation accuracy from that run reached 78.51 % validation and **86.67 % test** accuracy (a slightly earlier epoch than the last one, which is why it differs from the 84.17 % at epoch 30).

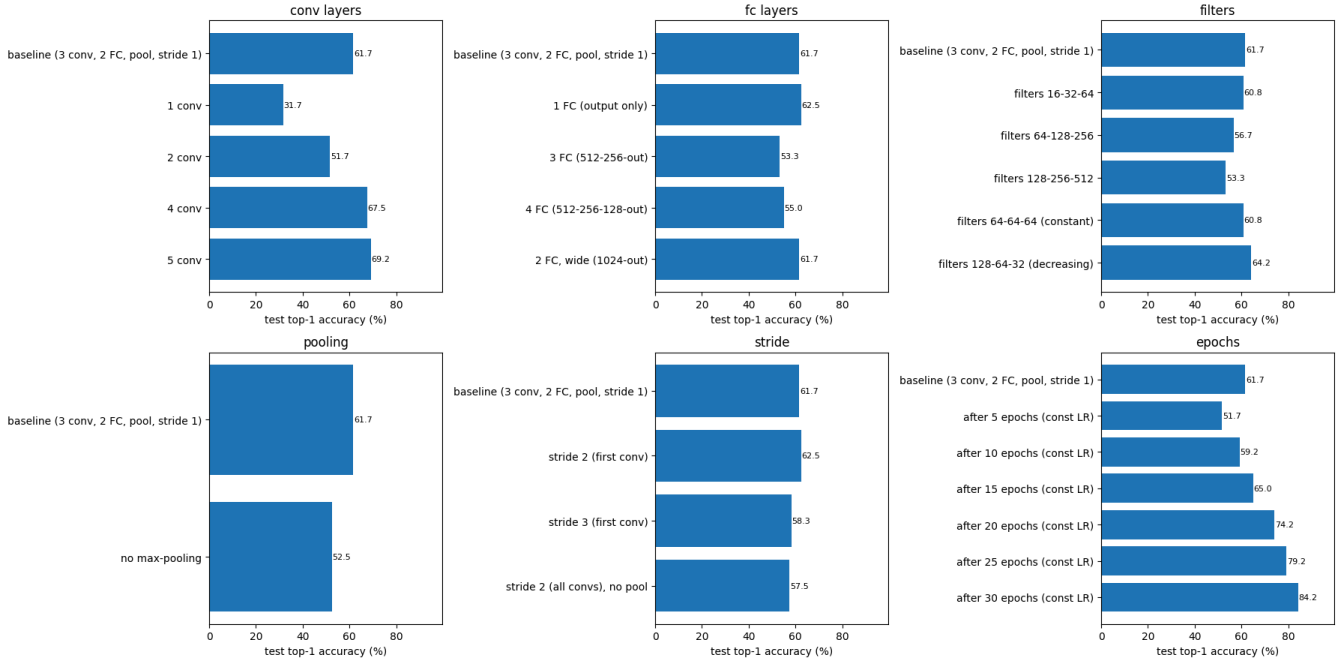



*Figure 1 – Test accuracy for each studied factor (file: fig_partA_ablation_bars.png).*

## 3.2 Observations, factor by factor

**Number of conv layers – the clearest effect.** Test accuracy rises steadily with depth: 31.7 % (1 conv) → 51.7 % → 61.7 % → 67.5 % → 69.2 % (5 conv), with top-5 accuracy saturating near 99 %. A single conv layer has a tiny receptive field and cannot tell apart shapes that differ in global structure (circle vs square vs triangle). Each added layer enlarges the receptive field and builds more abstract features. The gain shrinks at the end (4 → 5 layers is +1.7 points, about two test images, while the parameters grow by 40 %).

**Number of FC layers – no benefit.** A linear classifier directly on the conv features (1 FC, 0.13 M parameters) matches the baseline (62.5 % vs 61.7 %) with five times fewer parameters, and a wider hidden layer (1024) changes nothing. Adding hidden layers made results worse (53.3 % and 55.0 %). Training accuracy after 15 epochs was only 44.5 % for the 4-FC head against 74.4 % for the baseline, so the deeper head is slower to optimise (under-fitting within the fixed budget) rather than over-fitting. The convolutional layers do the real work; the FC head only needs to map features to classes.

**Number of filters – more is not better here.** The narrowest network (16-32-64, 0.29 M parameters) scored 60.8 %, essentially equal to the baseline, while the widest (128-256-512, 3.6 M parameters) scored 53.3 % and was about three times slower to train. Constant-width (60.8 %) and decreasing-width (64.2 %) stacks are also within noise. Validation and test rankings disagree (for example 64-128-256 has the best validation accuracy but a below-baseline test accuracy), which shows that these differences are within the noise of a 120-image test set. With only 959 training images, extra capacity does not help.

**Max-pooling – helpful and cheaper.** Removing the pooling layers lowered test accuracy from 61.7 % to 52.5 % (validation 64.5 % → 60.3 %) and made training about 2.7 × slower (8 s vs 3 s), because every layer then works on 84 × 84 maps. Pooling gives a small amount of translation invariance and reduces computation. The parameter count stays at 0.62 M in both cases because of the fixed 4 × 4 adaptive pooling before the FC head.

**Stride – free speed-up up to 2.** Using stride 2 in the first conv layer gave the same accuracy as the baseline (62.5 % vs 61.7 %) at roughly a third of the training time (about 1 s vs 3 s). Stride 3 lost about 3 points (58.3 %), and replacing pooling by stride 2 in all layers gave 57.5 %. Down-sampling early discards fine detail; here the objects are large (radius 12–26 px), so little is lost. On real photographs, with fine texture cues, a larger accuracy cost is expected and should be checked on the real data.

**Training time – the largest single effect.** With a constant learning rate, test accuracy grows from 51.7 % (5 epochs) to 84.2 % (30 epochs), and training accuracy reaches 96 % with the training loss still decreasing. The curves have not plateaued, so even longer training would probably help. The train/test gap widens (96 % vs 84 %), showing the beginning of over-fitting, but the test accuracy is still improving. In the 15-epoch comparison, the constant-LR run (65.0 %) and the cosine-schedule baseline (61.7 %) differ by about the noise level.

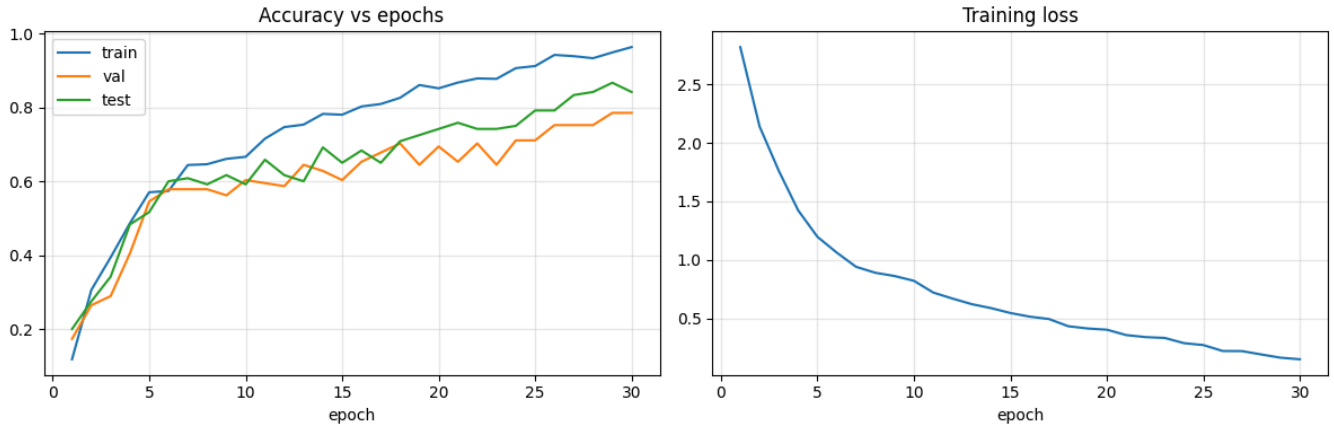



*Figure 2 – Accuracy and training loss over 30 epochs (file: fig_epochs_curve.png).*

## 3.3 Improving the performance

Three candidates combined the findings (more conv layers, VGG-style double convolutions, wider filters) with random flips and shifts, dropout 0.5, label smoothing 0.1 and 40 epochs with a cosine schedule. Among them, the 6-conv network (B) reached the best test accuracy (73.3 %) and the 8-conv network (C) the best validation accuracy (71.1 %), while the plain 4-conv network (A) reached 59.2 %. This again favours depth over width.

The best model **selected by validation accuracy over all runs**, however, was not one of these. It was the baseline architecture trained for 30 epochs without augmentation (validation 78.5 %, test 86.7 %). The improved candidates were still under-fitting after 40 epochs (training accuracy 61 %, 75 % and 67 % measured on augmented, label-smoothed batches, against 96 % for the 30-epoch run), so on this small, simple dataset the strong regularisation slowed learning more than it helped. This is consistent with the Part A result that training time is the most influential factor. It is not a general statement against augmentation: real Mini-ImageNet (60 000 images with much more variability) is expected to benefit from it, and this should be re-tested there. A fairer comparison would train the deeper 6-conv network for 30 or more epochs with and without augmentation.

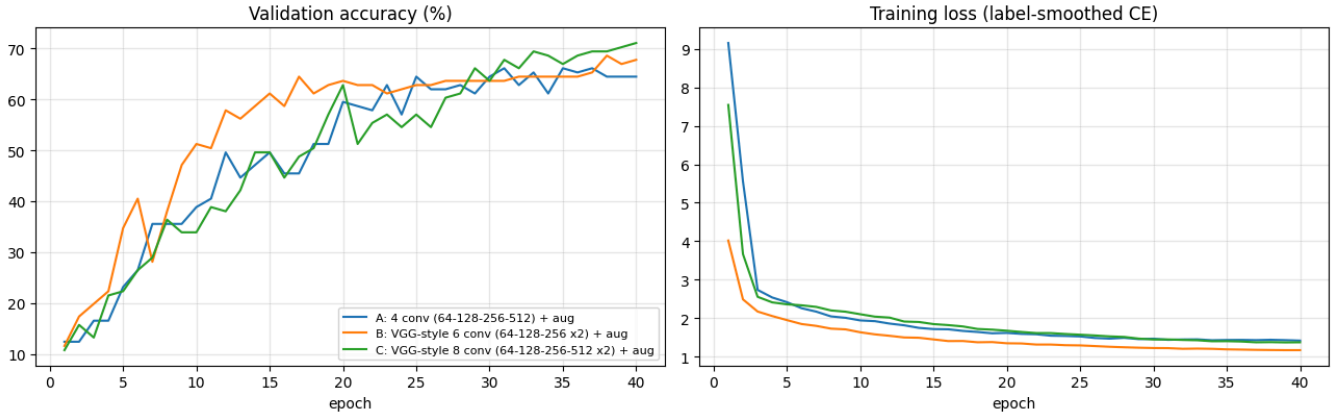


*Figure 3 – Validation accuracy and training loss of the improved candidates (file: fig_improved_curves.png).*

### 3.4 Final model

| Item | Value |
|---|---|
| Architecture | 3 conv (32-64-128, 3×3, stride 1) with 2×2 max-pooling, adaptive average pooling to 4×4, FC 256, dropout 0.3, output FC, BatchNorm |
| Training | AdamW, learning rate 1e-3 (constant), weight decay 5e-4, batch size 128, 30 epochs, no augmentation |
| Parameters | 0.62 M |
| Validation accuracy | 78.51 % |
| **Test accuracy** | **86.67 % top-1, 99.17 % top-5** |

## 4. Analysis of mis-classified images

The final model makes 16 errors on the 120 test images (13.3 %). For 93.8 % of these errors (15 of 16) the true class is still among the five highest-scoring predictions. The mean confidence is 0.85 for correct and 0.72 for wrong predictions, so the model is only modestly less sure when it is wrong, and some mistakes are made with very high confidence (up to 0.97).

**Which classes are confused.** The most frequent confusions are class_000 → class_001, class_012 → class_013, class_018 → class_016 and class_016 → class_017 (two images each). In every one of these pairs the true and predicted classes have the **same colour but a different shape**: circle ↔ square (000/001, 012/013, 016/017) and triangle → circle (018/016). Colour is easy to learn from three conv layers, but telling shapes apart, especially when the object is small, partly cut by the image border or has smoothed corners, is harder. The error pattern matches the Part A finding that depth (larger receptive field, more abstract shape features) is what limits accuracy.

**What the wrong images look like.** Many mis-classified objects are small (for example a small red triangle in the bottom-right corner of a class_002 image, or a small circle at the image edge), touch the border, or are circles/squares whose shape differs only slightly from the neighbour class once down-sampled. A few high-confidence errors are circles

predicted as squares of the same colour. Because the backgrounds are random noise, a context-based explanation is ruled out for this dataset; the errors are due to object size, position and shape similarity.

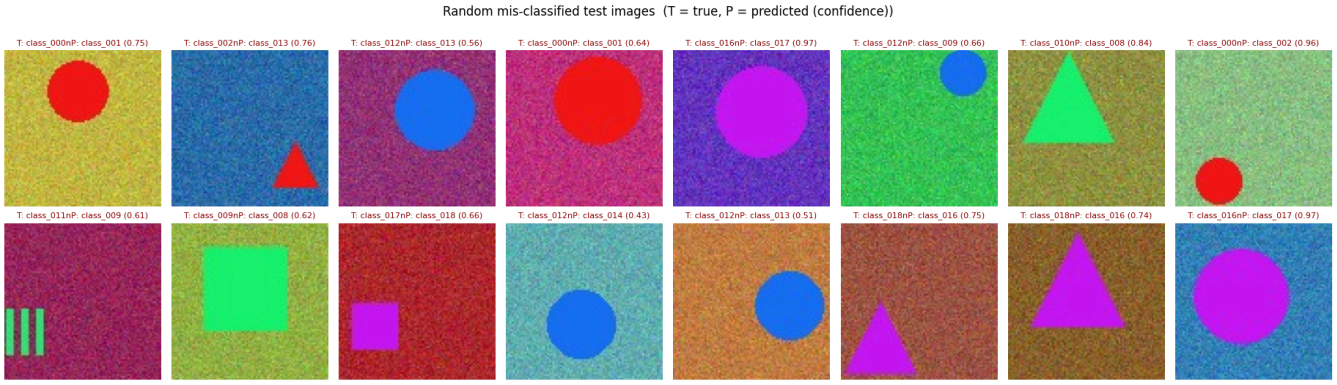
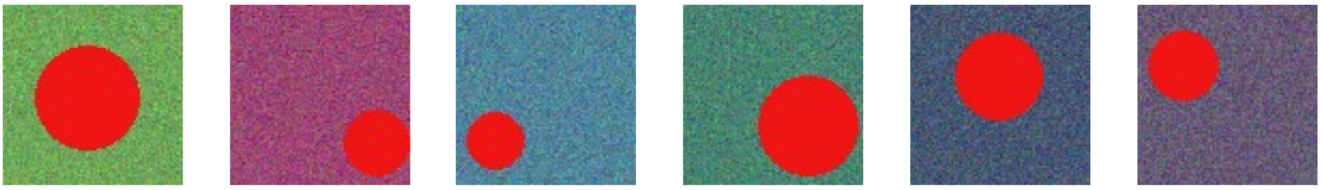
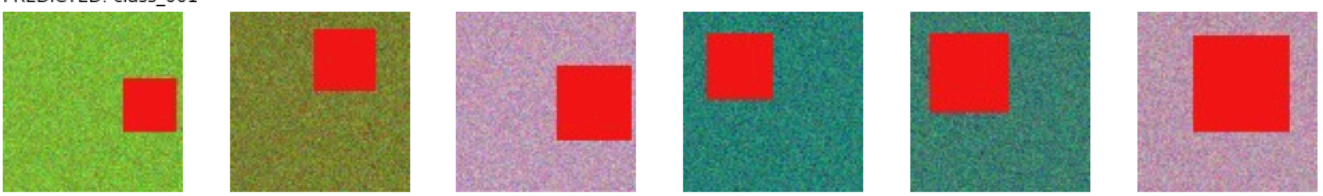



*Figure 4 – Randomly chosen mis-classified test images (T = true, P = predicted, confidence in brackets; file: fig_misclassified_random.png).*

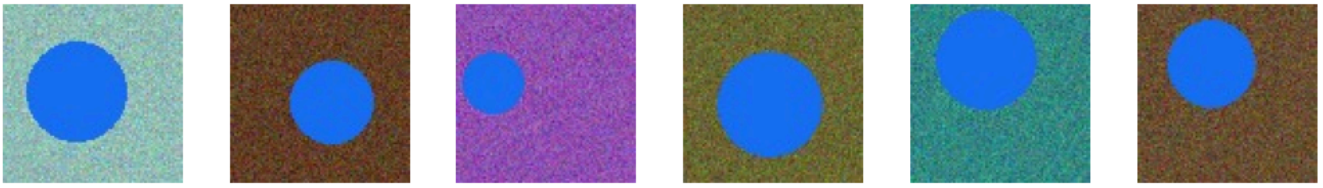
TRUE: class_000



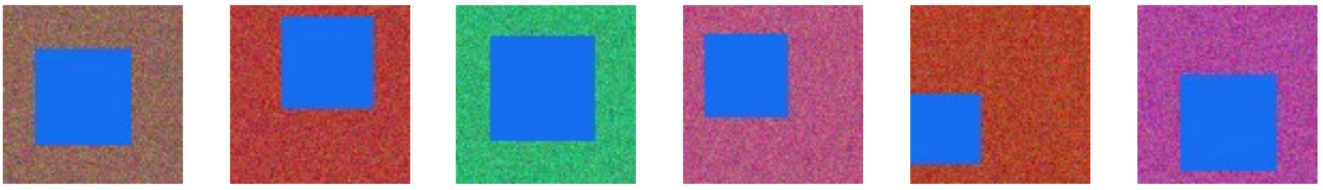
PREDICTED: class_001



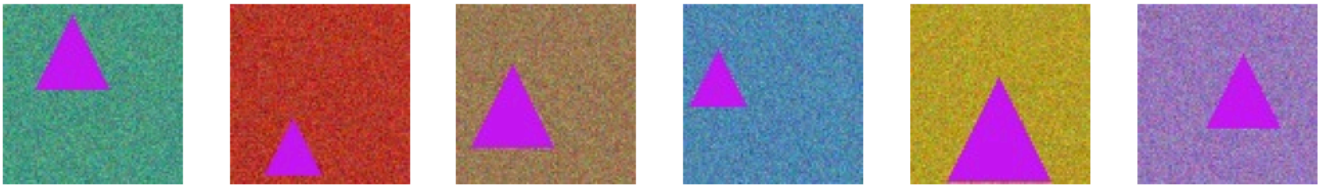
TRUE: class_012



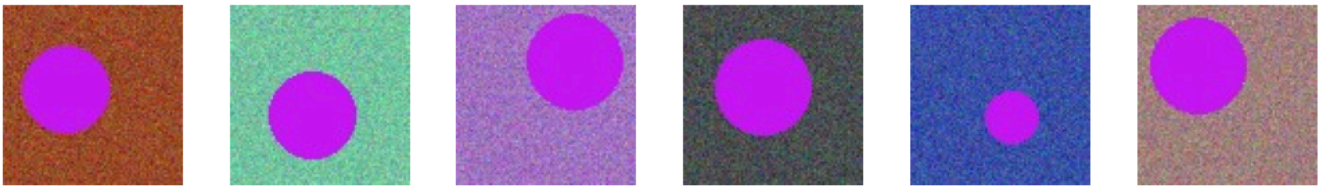
PREDICTED: class_013



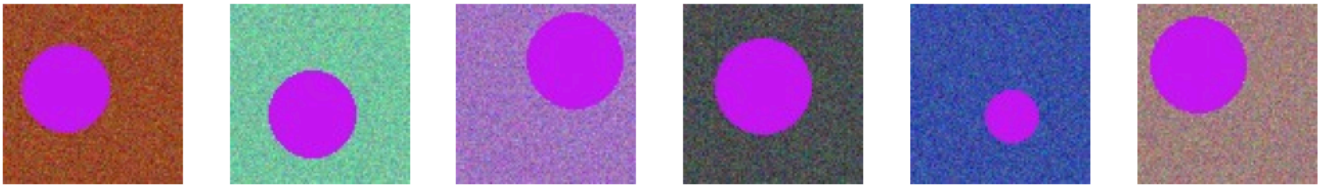
TRUE: class_018



PREDICTED: class_016



TRUE: class_016



PREDICTED: class_017

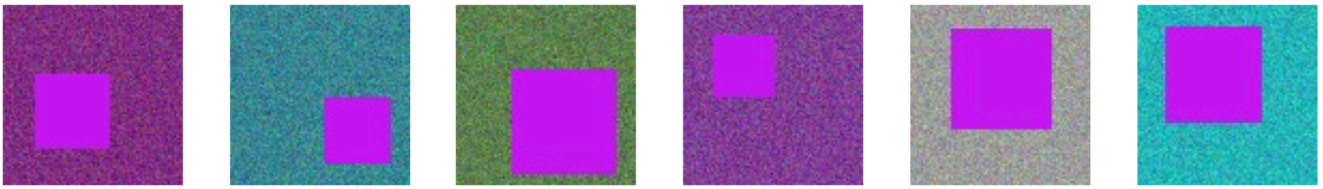


*Figure 5 – Training examples of the true class and the predicted class for the four most frequent confusions; the pairs differ only in shape (file: fig_confused_pairs_examples.png).*

**For the real Mini-ImageNet run**, expect confusions between visually similar fine-grained classes (for example dog breeds), images with several objects, small or partly visible objects, and cases where the background suggests another class; the same figures and tables can be used to discuss them.

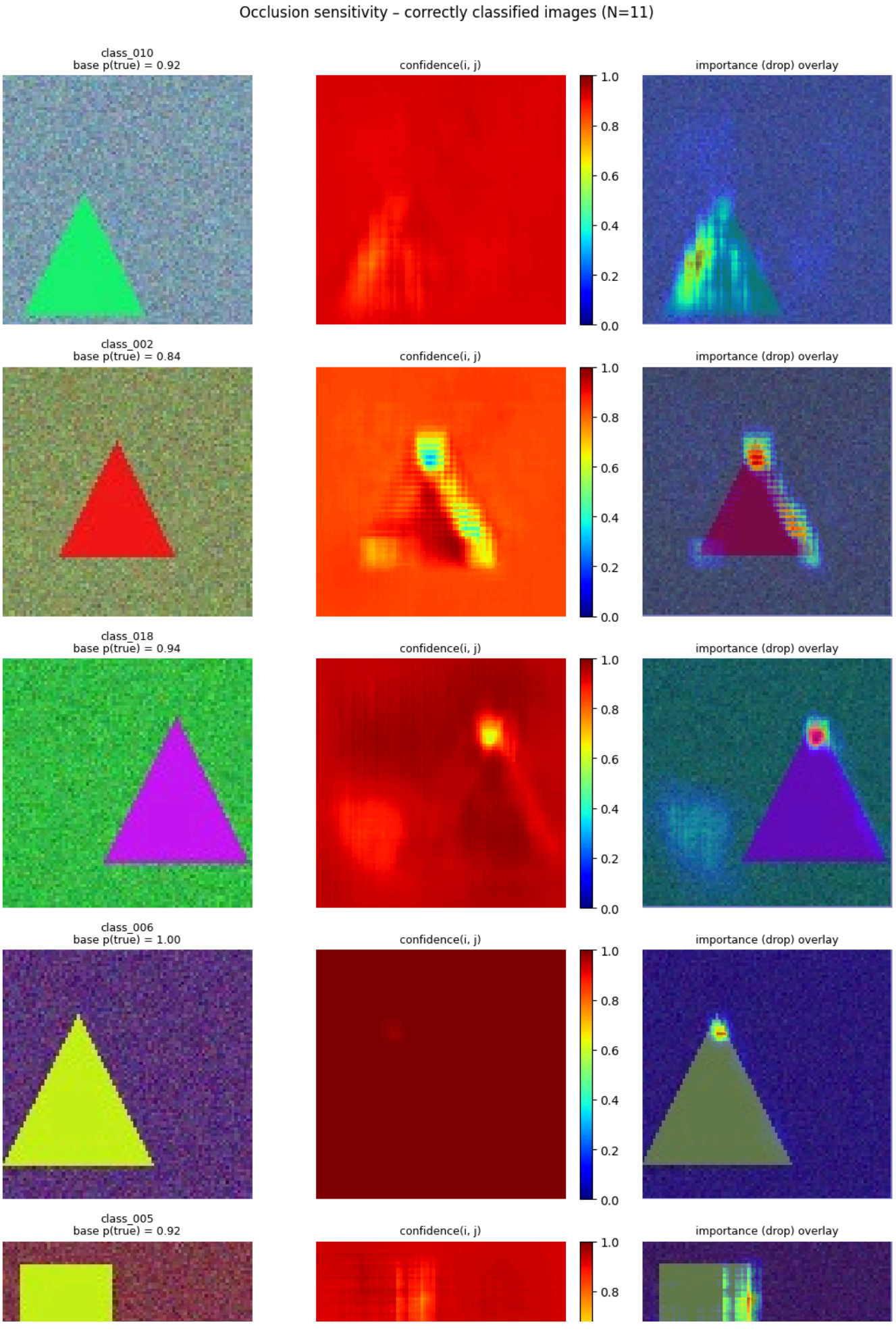
# 5. Part B – Occlusion sensitivity

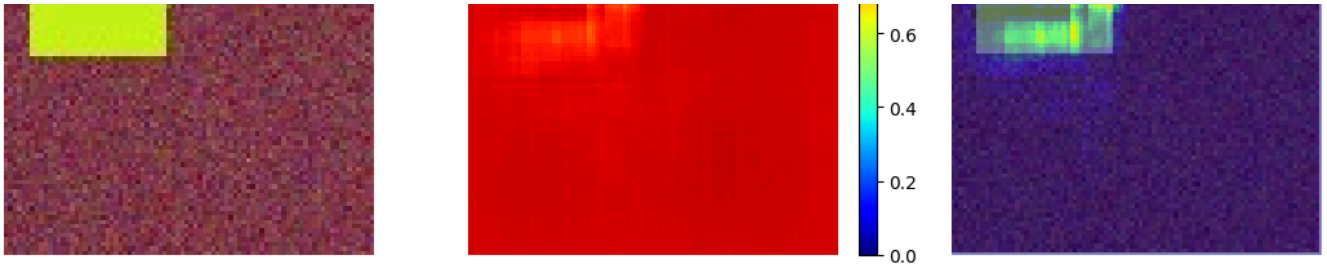
## 5.1 Method

The best model from Part A was used. For every pixel position (i, j) an N × N window centred on it (clipped at the image border) was replaced by gray pixels (value 0.5), the modified image was classified, and the probability of the **true** class was stored as confidence(i, j). With N = 11 (about 13 % of the image width) and stride 1 this requires 7,056 forward passes per image. Ten correctly classified test images from ten different classes (random, fixed seed) were analysed, plus four mis-classified images for comparison. A low value of confidence(i, j) means that hiding that location destroys the evidence for the true class.

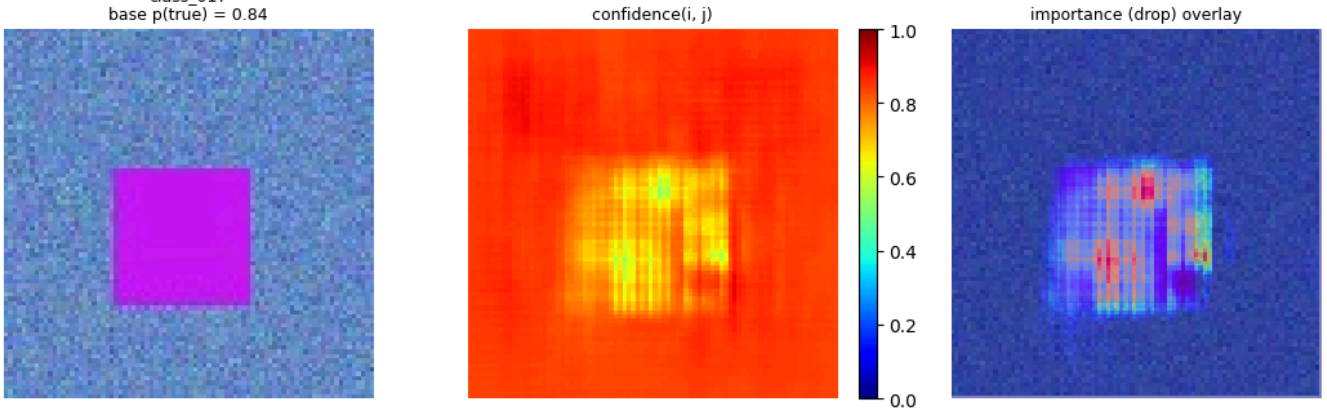
## 5.2 Results for correctly classified images

| Test image | Class | Base p(true) | Minimum p | % of positions with p < 0.5 × base | % of positions where prediction changes |
|---|---|---|---|---|---|
| 35 | class_010 | 0.918 | 0.776 | 0.0 | 0.0 |
| 78 | class_002 | 0.837 | 0.295 | 0.2 | 0.2 |
| 76 | class_018 | 0.945 | 0.597 | 0.0 | 0.0 |
| 64 | class_006 | 1.000 | 0.972 | 0.0 | 0.0 |
| 56 | class_005 | 0.923 | 0.749 | 0.0 | 0.0 |
| 108 | class_017 | 0.838 | 0.532 | 0.0 | 0.0 |
| 106 | class_007 | 0.987 | 0.932 | 0.0 | 0.0 |
| 11 | class_013 | 0.957 | 0.890 | 0.0 | 0.0 |
| 0 | class_009 | 0.740 | 0.210 | 2.8 | 10.2 |
| 20 | class_000 | 0.993 | 0.957 | 0.0 | 0.0 |
| **Mean** | | 0.91 | 0.69 | 0.3 | 1.0 |

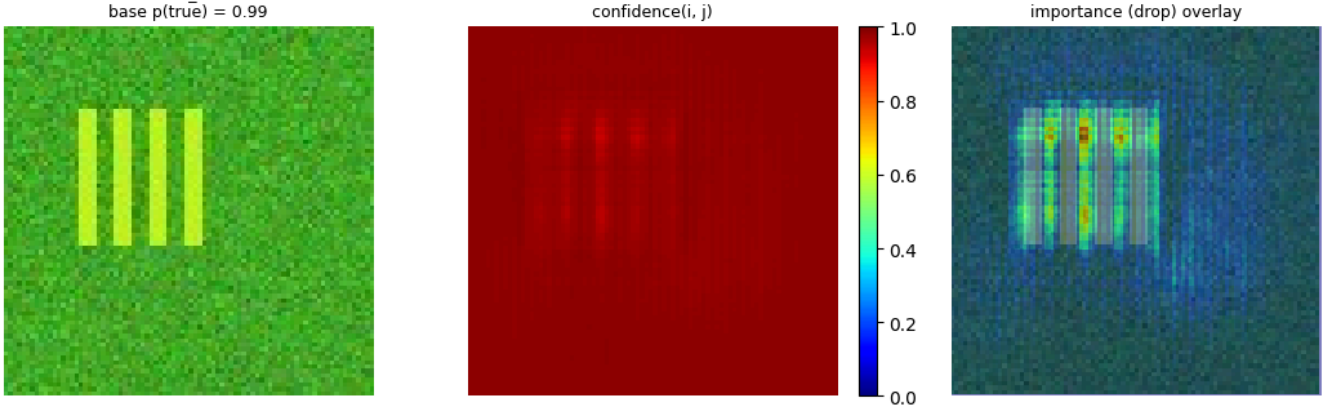

Occlusion sensitivity – correctly classified images (N=11)
class_010
base p(true) = 0.92
confidence(i, j)
importance (drop) overlay
class_002
base p(true) = 0.84
confidence(i, j)
importance (drop) overlay
class_018
base p(true) = 0.94
confidence(i, j)
importance (drop) overlay
class_006
base p(true) = 1.00
confidence(i, j)
importance (drop) overlay
class_005
base p(true) = 0.92
confidence(i, j)
importance (drop) overlay
1.0
0.8
0.6
0.4
0.2
0.0



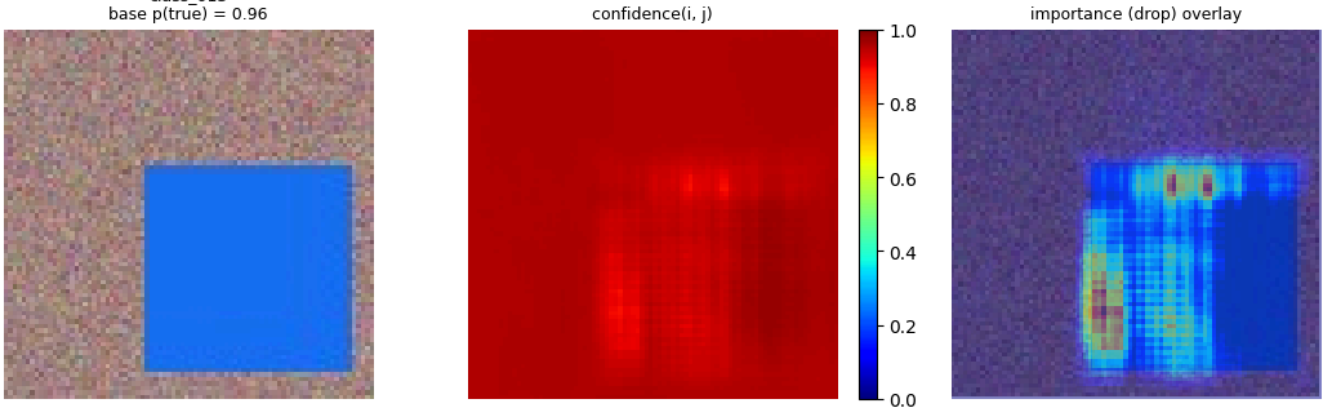

class_017
base p(true) = 0.84



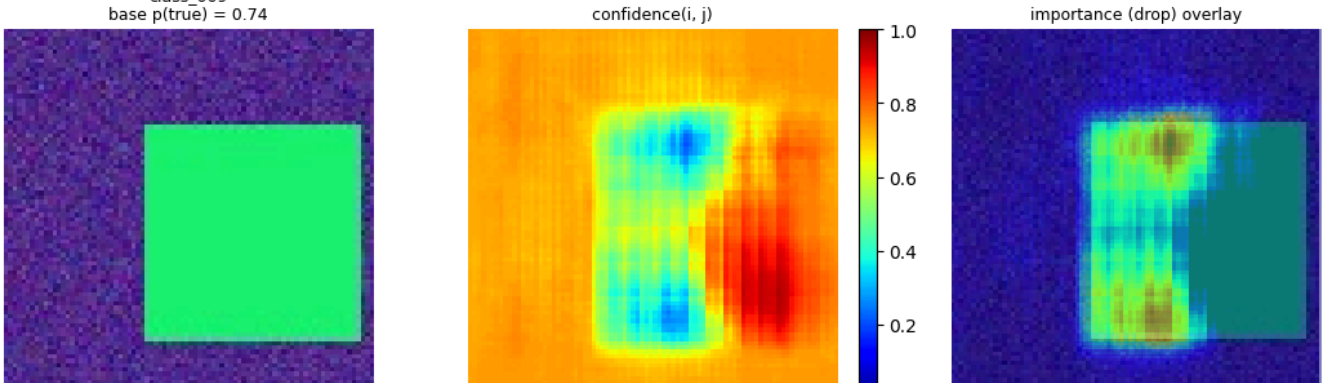

class_007
base p(true) = 0.99




class_013
base p(true) = 0.96




class_009
base p(true) = 0.74



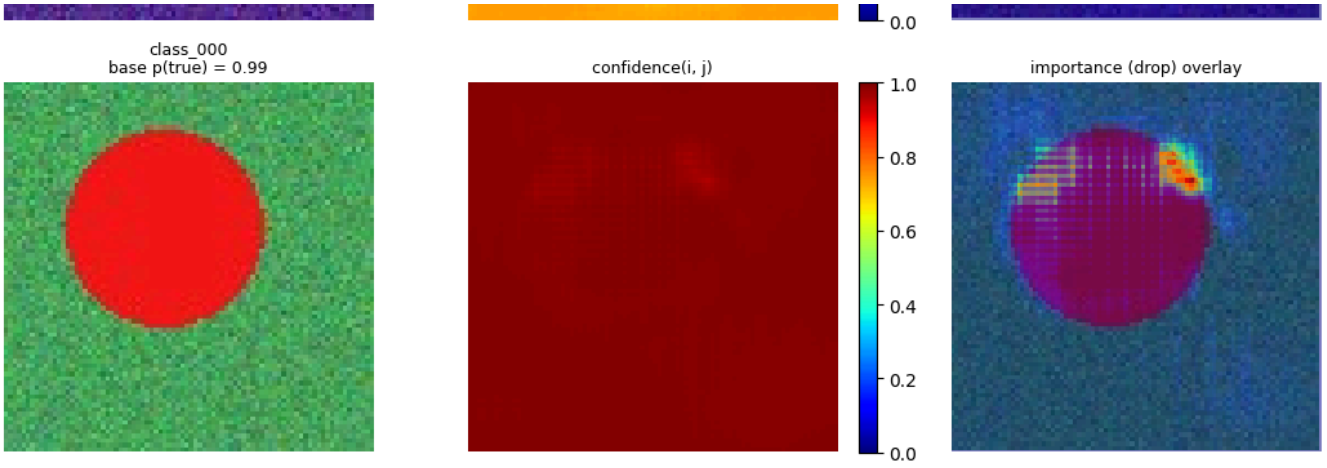



*Figure 6 – Occlusion maps for the ten correctly classified images: original, confidence(i, j) for the true class (colour scale 0–1) and an overlay of the confidence drop (file: fig_occlusion_correct.png).*

**Observations.**

- **The model looks at the object, not the background.** In all ten images the drop in confidence is located on or directly around the object. Hiding any part of the background leaves the confidence unchanged (the flat red areas of the maps).
- **It relies on diagnostic local features.** Sensitivity is concentrated on triangle apexes and edges (classes 002, 006, 010, 018), square edges and corners (005, 013, 017), the stripe pattern (007) and the circle contour (000), rather than on the flat interior of the shape. Hiding the uniform interior costs very little.
- **The effect of an 11 × 11 window is small.** On average only 0.3 % of the positions lower the confidence below half of its base value, and three images never fall below 0.93. The most confident images (base p ⩾ 0.99: classes 006, 007, 000) are almost indifferent to such a small occluder, which indicates redundant evidence. The most fragile ones (class_009, minimum 0.21; class_002, minimum 0.30) had lower base confidence, and for class_009 the prediction changes at 10 % of the positions.
- **Average behaviour over 103 correctly classified images** (N = 11, stride 4): the mean relative confidence is lowest in the central region (about 0.94 at its minimum, 0.958 averaged over the central third) and about 0.997 at the borders. This should not be read as a centre bias of the model: in the dummy data the object centre is drawn uniformly inside the image, so objects cover the central region more often than the borders. On real data, where objects tend to be centred, this analysis should be repeated.

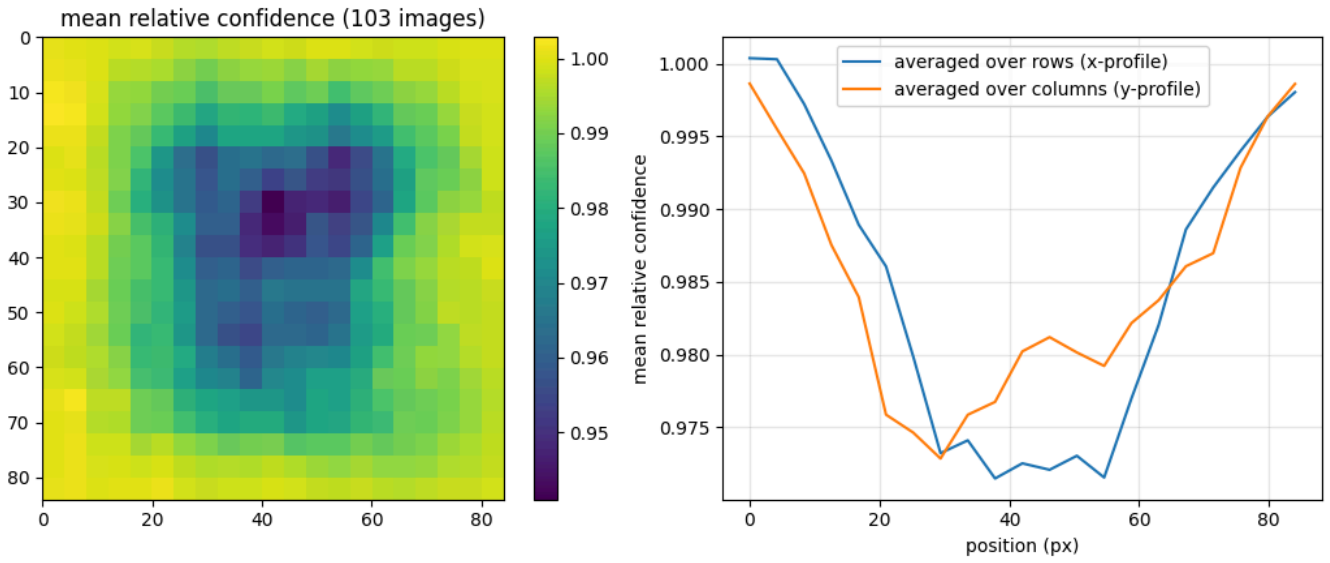



*Figure 7 – Average relative confidence map over 103 correctly classified images and its x/y profiles (file: fig_occlusion_average_map.png).*

## 5.3 Effect of the window size

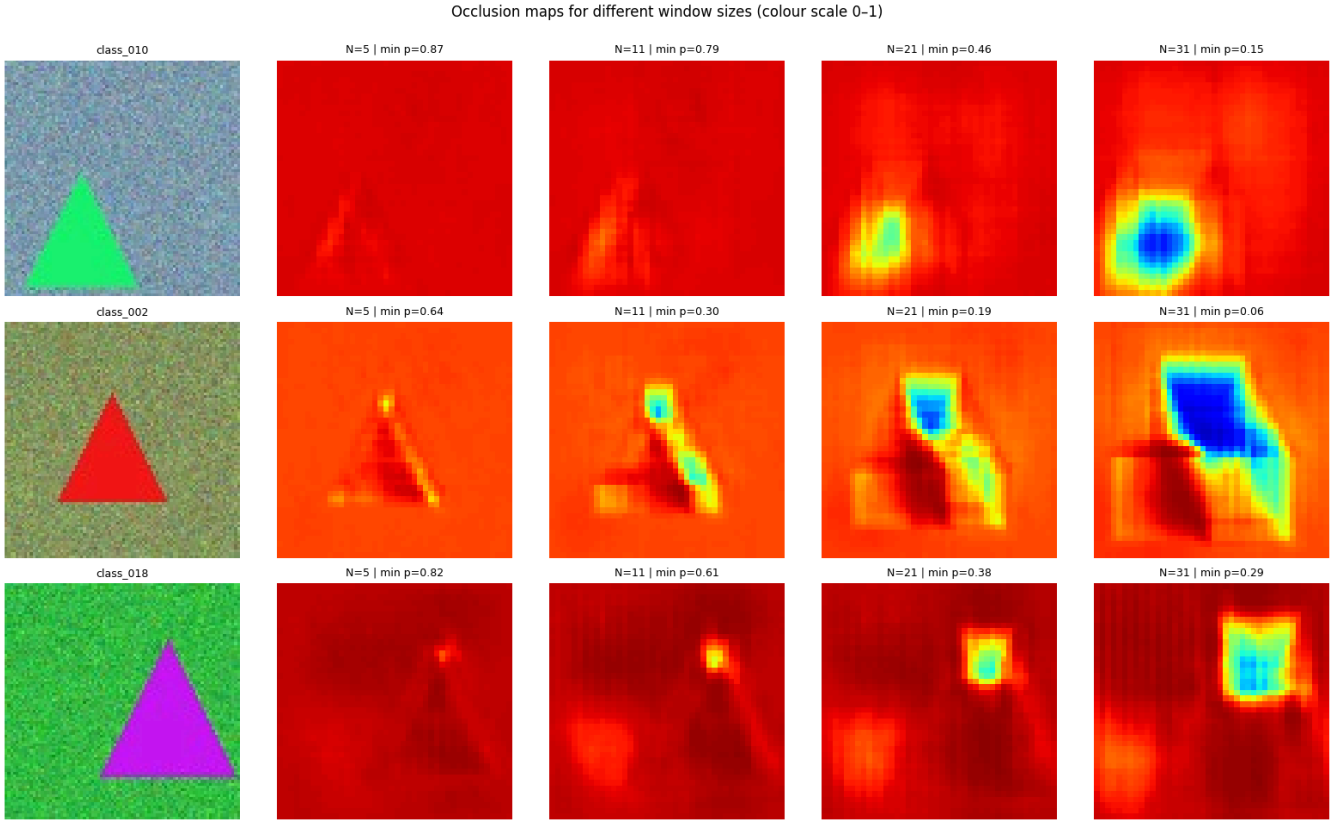



*Figure 8 – Occlusion maps of three images for N = 5, 11, 21 and 31 (file: fig_occlusion_window_sizes.png).*

The window size controls what is being tested. With N = 5 the minimum confidence stays high (0.64–0.87) and only thin edge features appear. N = 11 localises the key features (apex, edges). With N = 21 and 31 the occluder covers entire features and the confidence

falls to 0.06–0.46, but the maps become blurry blobs about as large as the window. N = 11 is a good compromise between localisation and sensitivity for 84 × 84 images.

## 5.4 Mis-classified images

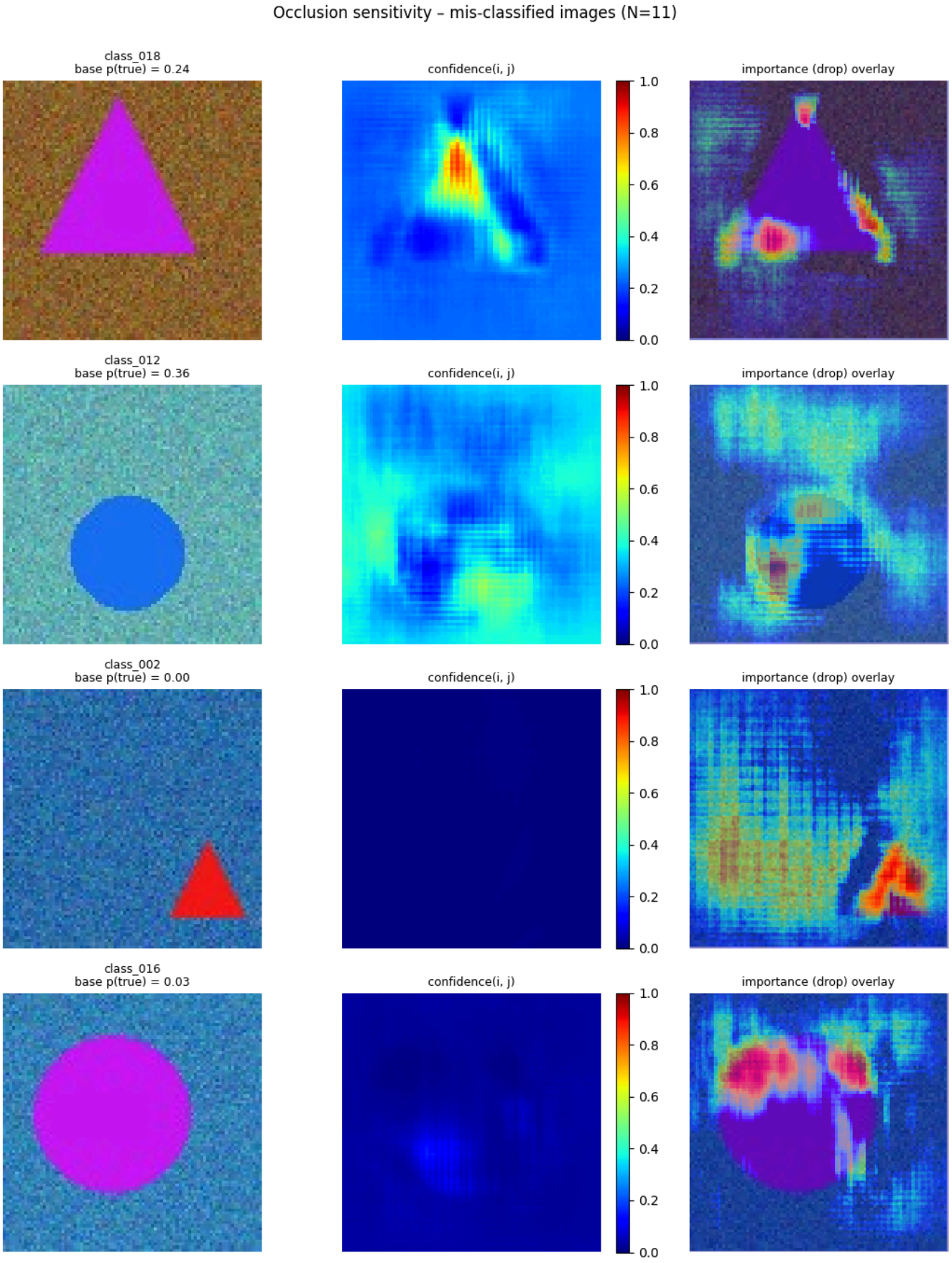



*Figure 9 – Occlusion maps for four mis-classified images; confidence is still the probability of the true class (file: fig_occlusion_wrong.png).*

For the four wrong images the true-class probability is low everywhere (base values 0.24, 0.36, 0.004 and 0.03). For class_018 (triangle predicted as a circle of the same colour), hiding a small region in the upper part of the triangle raises the true-class probability from 0.24 to roughly 0.7, so the wrong decision depended on one local region. For the other three images no occluded window helps: the evidence for the true class is weak across the whole image (for example a small triangle in the corner of a class_002 image), so these are failures to recognise the object rather than effects of a misleading background.

### 5.5 Conclusion of Part B

The model has learnt where the object is and bases its decision on object shape features such as corners and edges; it does not depend on the background. This outcome is expected for the synthetic data, because the backgrounds are random noise and carry no class information. The experiment on real Mini-ImageNet is the one that can reveal context dependence (for example water for boats or grass for animals), and the same code and interpretation guide apply.

## 6. Summary and conclusions

- **Depth matters most among the architecture choices.** Test accuracy increased from 31.7 % to 69.2 % when going from 1 to 5 conv layers, with diminishing returns after 4 layers.
- **More FC layers and more filters did not help** (differences within noise or negative) while increasing cost. A linear head on the conv features was as good as the baseline with five times fewer parameters.
- **Max-pooling helps** (+9 points here and 2.7 × faster); **stride 2 in the first layer is a free speed-up**, stride 3 loses accuracy.
- **Training time was the largest single factor:** 51.7 % after 5 epochs to 84.2 % after 30, with no plateau. The best model (3 conv, 0.62 M parameters, 30 epochs) reached **86.67 % top-1 and 99.17 % top-5** test accuracy.
- **The error analysis** shows that mistakes are mostly between classes that share colour and differ in shape, and involve small or border-clipped objects; 94 % of the errors still have the true class in the top five.
- **The occlusion study** shows that the model uses the object, especially its corners and edges, and ignores the background; the 11 × 11 window is a suitable size.

**Limitations.** The results come from a small synthetic dataset with a 120-image test set and single-seed runs, so only large differences are reliable. Future work: repeat on the real Mini-ImageNet data, run several seeds and report confidence intervals, train the deeper networks for equally long with and without augmentation, and test whether context dependence appears in the occlusion maps on natural images.

# References

1. M. D. Zeiler and R. Fergus, "Visualizing and understanding convolutional networks," European Conference on Computer Vision, pp. 818–833, Springer, 2014.
2. Mini-ImageNet dataset: https://github.com/yaoyao-liu/mini-imagenet-tools

# Appendix – Files delivered with the report

Code: `DL_Module4_MiniImageNet_CNN_Occlusion.ipynb`